

Lab Practice Problem for Final Exam

Filename: No Submission Required

In this activity, you will review a representative set of questions covering the major topics discussed in the course. These problems are designed to help you evaluate your understanding of key concepts and strengthen your problem-solving skills.

Important: Please note that this collection is not exhaustive. It serves only as a sample of the types of questions you may encounter on the exam. You are expected to understand the fundamental principles behind each topic and be prepared to apply them in a variety of scenarios.

Time and Point Overview

This practice set is designed to be completed in approximately **50 minutes** under exam conditions. Actual completion time may vary depending on familiarity and level of preparation. You are encouraged to practice pacing and time management.

Topic	Points	Estimated Time
Summation	8	3–4 minutes
Recurrence Relations	10	5–6 minutes
Sorting Algorithms	12	6–8 minutes
Binary Trees	10	5–6 minutes
AVL Trees	10	5–6 minutes
Binary Heaps	10	5–6 minutes
Tries	10	5–6 minutes
Hash Tables	10	5–6 minutes
Base Conversion	8	3–4 minutes
Bitwise Operators	12	5–6 minutes
Total	100	50 minutes

Note: While it is possible to complete all problems within the 50-minute time limit, it is tight. Focus on pacing, accuracy, and prioritizing problems effectively.

Summation

Determine a closed-form solution for the summation shown below, in terms of n :

$$\sum_{i=1}^n \sum_{j=1}^i j$$

Solve this problem independently; it is not covered in the lab. A complete solution is provided at the end of the document.

Recurrence Relations

Given the following implementation of merge sort, write the recurrence relation $T(n)$ for the running time, assuming the function is sorting an array of size n .

```
void mergeSort(int *arr, int left, int right) {  
    if(left >= right)  
        return;  
  
    int mid = (left + right) / 2;  
  
    mergeSort(arr, left, mid);  
    mergeSort(arr, mid + 1, right);  
    merge(arr, left, mid, right);  
}
```

$$T(n) = 2T\left(\frac{n}{2}\right) + n \quad \text{for } n > 1$$
$$T(1) = 1$$

Solve the recurrence using the iterative method. Show at least two iterations of the expansion, then express the overall running time using Big-Oh notation.

Solve this problem independently; it is not covered in the lab. A complete solution is provided at the end of the document.

Sorting Algorithms

Consider the following array:

Index	0	1	2	3	4	5	6	7
Original	80	30	20	50	60	70	10	40

For each of the following, begin with the original array shown above. Do not use the result of a previous part. Each question is independent.

Show the state of the entire array after one iteration of the outer loop of **selection sort**.

10, 30, 20, 50, 60, 70, 80, 40

Intuition: Find the smallest element in the array and swap it with the element at index 0.

Show the state of the entire array after one iteration of the outer loop of **insertion sort**.

30, 80, 20, 50, 60, 70, 10, 40

Intuition: Take the element at index 1 and insert it into its correct position within the sorted portion of the array (i.e., the left side).

Show the state of the entire array after one iteration of the outer loop of **bubble sort**.

30, 20, 50, 60, 70, 10, 40, 80

Intuition: Repeatedly compare and swap adjacent elements. After one full pass, the largest element moves to its correct position at the far right.

Show the state of the entire array after performing the first two merge operations in **merge sort**.

30, 80, 20, 50, 60, 70, 10, 40

Intuition: Merge operations are performed on the left subarray first. Since the recursive calls execute sequentially (not in parallel), the first merge combines the elements at indices 0 and 1, followed by the second merge combining the elements at indices 2 and 3.

Binary Trees

Consider the implementation of a binary tree:

```
typedef struct BTNode_s {  
    int data;  
    struct BTNode_s *left;  
    struct BTNode_s *right;  
} BTNode;
```

Complete the function that returns the number of internal nodes in a binary tree. Recall that an internal node is a node with at least one child.

```
int count_internal(BTNode *root) {
```

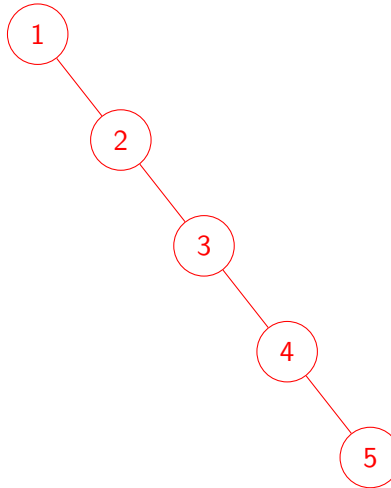
```
    if( root == NULL )  
        return 0;  
  
    // General strategy: Similar to LRV traversal  
  
    // Get the count of both children: LR  
    int left = count_internal( root->left );  
    int right = count_internal( root->right );  
  
    // Get the running total  
    int count = left + right;  
  
    // If the current node meets the criteria, add it: V  
    if( root->left != NULL || root->right != NULL )  
        count++;  
  
    return count;
```

```
}
```

AVL Trees

Draw a Binary Search Tree (BST) that stores the values 1, 2, 3, 4, and 5 such that the tree is a valid BST but **not** a valid AVL tree.

Here is one possible example:



Ensure that the BST property holds for every node: all values in the left subtree are less than the node, and all values in the right subtree are greater than the node. To make the tree **not** an AVL tree, ensure that at least one node violates the balance condition, meaning its balance factor is not -1 , 0 , or $+1$.

State the additional condition that a BST must satisfy in order to be considered an AVL tree.

The balance factor of every node, defined as the difference between the heights of its left and right subtrees, must be either -1 , 0 , or $+1$.

Binary Heaps

Write a function `heapDelete()` that removes an element at a given index from a min-heap. The function should accept the following parameters: the heap array, a pointer to the current number of elements in the heap, and the index of the element to be deleted. Assume that the root of the heap is stored at index 0. If the index is invalid, the function should return 0. Otherwise, the function removes the element, restores the heap property, and returns 1.

Assume the following helper functions are already implemented. These functions are defensive, meaning they safely handle invalid indices and perform any necessary bounds checking internally.

```
void percolateUp(int arr[], int arrSize, int upIndex);
// Performs a percolate-up operation starting at index upIndex.
// Note: arrSize refers to the number of items (i.e., heapSize).

void percolateDown(int arr[], int arrSize, int downIndex);
// Performs a percolate-down operation starting at index downIndex.
// Note: arrSize refers to the number of items (i.e., heapSize).
```

Important Note

Percolate-up corresponds to the operation referred to in class as heapify-up, while percolate-down corresponds to heapify-down.

```
int heapDelete(int heapArr[], int *heapSize, int idx) {
```

```
    if( idx < 0 || idx >= *heapSize ) return 0;

    // Decrement the size of the heap
    *heapSize = *heapSize - 1;        // or (*heapSize)--;    be careful!

    // If removing the last position element
    if( idx == *heapSize ) return 1;

    // Remember the old value
    int tmp = heapArr[idx];

    // Replace the element at idx with the last element
    heapArr[idx] = heapArr[*heapSize];

    // If we replaced idx with a lower value, go up; otherwise, go down
    if( heapArr[idx] < tmp )
        percolateUp( heapArr, *heapSize, idx );
    else
        percolateDown( heapArr, *heapSize, idx );

    return 1;
```

```
}
```

Tries

Consider the implementation of a trie:

```
typedef struct TrieNode_s {
    struct TrieNode_s *children[26];
    int is_end_of_word;
} TrieNode;
```

Complete the function that returns the number of words in the trie whose length is exactly `len`. The function takes the root of the trie and the value `len` as parameters.

```
int count_words(TrieNode *node, int len) {
```

```
    // If there is no node, then there is no word
    if( node == NULL )
        return 0;

    // We are now at the level that is len characters away from the root
    // The last step is to check if this node is marked as end_of_word
    if( len == 0 )
        return node->is_end_of_word;

    // This will keep track of the running total
    int count = 0;

    // Add the number of words found for each of the 26-children paths
    for(int i = 0; i < 26; i++)
        count = count + count_words( node->children[i], len-1 );

    return count;
```

```
}
```


Hash Tables

Consider a hash table of size $m = 11$ that uses **quadratic probing** for collision resolution. The hash function is defined as:

$$f(x) = (5x + 2) \% 11$$

Hint: The symbol % denotes the modulo operation, which returns the remainder after division.

Insert the following values into the hash table in the given order:

3, 9, 2, 1, 14, 6, 25

Index	0	1	2	3	4	5	6	7	8	9	10
Value											

Index	0	1	2	3	4	5	6	7	8	9	10
Value	6	2		9	25		3	1			14

Tip: First, precompute all hash values so you can focus entirely on the insertion step. Since quadratic probing is used, generate the sequence of perfect squares in advance (1, 4, 9, 16, ...). When a collision occurs, move to the next value in this sequence. This approach eliminates the need to restart the count each time you search for the next available slot.

What is the main issue associated with **linear probing** in hash tables?

Linear probing is prone to **primary clustering**. When a collision occurs, the algorithm searches for the next available slot in sequence, leading to contiguous blocks of occupied cells.

Base Conversion

Convert the following binary number to decimal:

$$101101_2$$

Assign powers of 2 to each bit position from right to left:

$$1, 2, 4, 8, 16, 32$$

Select the values corresponding to the bits that are 1 in 101101_2 :

$$1, 4, 8, 32$$

Add them together:

$$1 + 4 + 8 + 32 = 45$$

$$\boxed{45_{10}}$$

Convert the following decimal number to hexadecimal:

$$127_{10}$$

Observe that 127 is close to 128. In binary, 128 is:

$$10000000_2$$

Subtracting 1 gives:

$$127 = 01111111_2$$

Next, group the binary number into sets of four bits from right to left:

$$0111 \ 1111$$

Finally, convert each group to hexadecimal:

$$0111 = 7, \quad 1111 = F$$

$$\boxed{127_{10} = 7F_{16}}$$

Bitwise Operator

An integer `sample` is said to contain a virus if the rightmost 8 bits of its binary representation contain at least `n` bits set to 1, where $n \leq 8$. Complete the function that returns 1 if the sample contains the virus, and 0 otherwise.

Example

Let sample = 14. Its 32-bit binary representation is:

[illegible]

The rightmost 8 bits are:

00001110

If we suppose that:

- $n = 2$, the rightmost 8 bits contain 3 ones, which is ≥ 2 . Therefore, the function should return 1.
- $n = 4$, the rightmost 8 bits contain only 3 ones, which is < 4 . Therefore, the function should return 0.

```
int hasVirus(int sample, int n) {
```

```
// Counter for the number of 1 bits
int c = 0;

// Inspect the 8 rightmost bits
for(int i = 0; i < 8; i++) {
    // Check if the rightmost bit is a 1
    if( ( sample & 1 ) == 1 )
        c++;

    // Perform a right shift
    sample = sample >> 1;
}

// Check if the counter is at least n bits
return c >= n;
```

}

Solution: Summation

We are given the summation:

$$\sum_{i=1}^n \sum_{j=1}^i j$$

Step 1: Evaluate the inner summation.

For a fixed i , we know:

$$\sum_{j=1}^i j = \frac{i(i+1)}{2}$$

Step 2: Substitute into the outer summation.

$$\sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \sum_{i=1}^n (i^2 + i)$$

Step 3: Split the summation.

$$= \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right)$$

Step 4: Use known formulas.

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Step 5: Substitute.

$$= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right)$$

Step 6: Factor and simplify.

$$\begin{aligned} &= \frac{1}{2} \cdot n(n+1) \left(\frac{2n+1}{6} + \frac{1}{2} \right) \\ &= \frac{1}{2} \cdot n(n+1) \left(\frac{2n+1+3}{6} \right) \\ &= \frac{1}{2} \cdot n(n+1) \cdot \frac{2n+4}{6} \\ &= \frac{n(n+1)(n+2)}{6} \end{aligned}$$

Final Answer:

$$\sum_{i=1}^n \sum_{j=1}^i j = \frac{n(n+1)(n+2)}{6}$$

Solution: Recurrence Relations

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(1) = 1$$

k	$T(n)$	Worksheet
1	$T(n) = 2T\left(\frac{n}{2}\right) + n$ $T(n) = 2T\left(\frac{n}{2}\right) + n$ $= 2\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right] + n$	$T\left(\frac{n}{2}\right) = 2T\left(\frac{\frac{n}{2}}{2}\right) + \frac{n}{2}$ $= 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}$
2	$T(n) = 2^2T\left(\frac{n}{2^2}\right) + n + n$ $T(n) = 2^2T\left(\frac{n}{2^2}\right) + n + n$ $= 2^2\left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right] + n + n$	$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{\frac{n}{2^2}}{2}\right) + \frac{n}{2^2}$ $= 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}$
3	$T(n) = 2^3T\left(\frac{n}{2^3}\right) + n + n + n$	
k	$T(n) = 2^kT\left(\frac{n}{2^k}\right) + kn$	

Base Case:

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$\log n = \log(2^k)$$

$$\log n = k \log 2$$

$$\frac{\log n}{\log 2} = k$$

$$\log_2 n = k$$

Substitute back:

$$\begin{aligned}T(n) &= 2^k T\left(\frac{n}{2^k}\right) + kn \\&= 2^k T(1) + kn \quad \text{since } \frac{n}{2^k} = 1 \\&= \left(2^{\log_2 n}\right) T(1) + n \log_2 n \\&= n \cdot T(1) + n \log_2 n \\&= n \cdot (1) + n \log_2 n \\&= n + n \log_2 n\end{aligned}$$

Final Answer:

$$T(n) = O(n \log n)$$